

Финансовый университет при Правительстве РФ

КУРСОВАЯ РАБОТА

по дисциплине: _____

**на тему: «Разработка автоматической диалоговой системы на основе
языковой модели для анализа экономических новостей»**

Выполнил: Прудиев Алексей Сергеевич

Группа: ПМ23-4

Руководитель: _____

Москва, 2026

СОДЕРЖАНИЕ

1. Введение
2. Анализ предметной области
3. Проектирование диалоговой системы
4. Реализация программного проекта
5. Тестирование и демонстрация результатов
6. Заключение
7. Список использованных источников

ВВЕДЕНИЕ

Экономические новости ежедневно влияют на ожидания инвесторов, компаний, потребителей и государственных органов. Рост ВВП, изменение инфляции, решения центральных банков, динамика безработицы и торговые показатели могут менять оценку рыночных рисков.

Автоматические диалоговые системы позволяют упростить работу с такими данными: пользователь формулирует вопрос естественным языком, а система находит релевантные источники, анализирует их и формирует краткий ответ.

Работа находится на стыке машинного обучения и backend-разработки. ML/NLP-часть отвечает за текстовое представление новостей, подготовку датасетов, поиск релевантного контекста, обучение классификаторов влияния и подключение языковой модели. Инженерная часть отвечает за микросервисную архитектуру, асинхронную обработку, загрузку CSV через интерфейс, веб-интерфейс и воспроизводимый запуск.

Объектом работы являются экономические новости. Предметом работы являются методы построения автоматической диалоговой системы для поиска, анализа и обобщения экономических новостей.

Цель работы: разработать локальную автоматическую диалоговую систему, которая принимает вопрос пользователя, находит релевантные

экономические новости, оценивает их влияние и формирует ответ на основе найденных источников.

Для достижения цели поставлены задачи:

1. Изучить предметную область анализа экономических новостей.
2. Определить ML/NLP pipeline: представление текста, подготовка датасета, поиск, классификация влияния и генерация ответа.
3. Спроектировать микросервисную архитектуру диалоговой системы.
4. Реализовать сервис загрузки и индексации новостей.
5. Реализовать сервис векторного поиска релевантных новостей.
6. Реализовать сервис анализа влияния новости.
7. Подготовить CLI-сценарий преобразования внешнего CSV и обучения классификаторов.
8. Реализовать сервис генерации ответа с поддержкой языковой модели.
9. Реализовать веб-интерфейс для загрузки датасета и работы пользователя.
10. Подготовить воспроизводимый сценарий запуска и проверки.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Экономические новости представляют собой текстовые сообщения о событиях, которые могут влиять на рынок, отрасли и отдельные компании. Такие события часто связаны с макроэкономическими показателями, монетарной политикой, торговлей, занятостью и промышленностью.

С точки зрения машинного обучения задача включает несколько подзадач обработки естественного языка: нормализацию и представление текста, поиск релевантных документов, классификацию влияния новости и генерацию итогового ответа. Поэтому система должна не только отдавать HTTP-ответ, но и выполнять понятный ML/NLP pipeline.

Для классификации влияния новости применяется воспроизводимый baseline tfidf-logreg. Такой подход хорошо подходит для учебной защиты: он объясним, быстро запускается локально и показывает связь между

текстовыми признаками и итоговым классом влияния. Для исследовательского сценария также предусмотрены подготовка размеченного CSV, обучение режима на embeddings и обучение небольшой transformer-модели.

Retrieval-подход позволяет сначала найти контекст, а затем формировать ответ на основе найденных документов. Это снижает риск абстрактного ответа и делает результат объяснимым для пользователя.

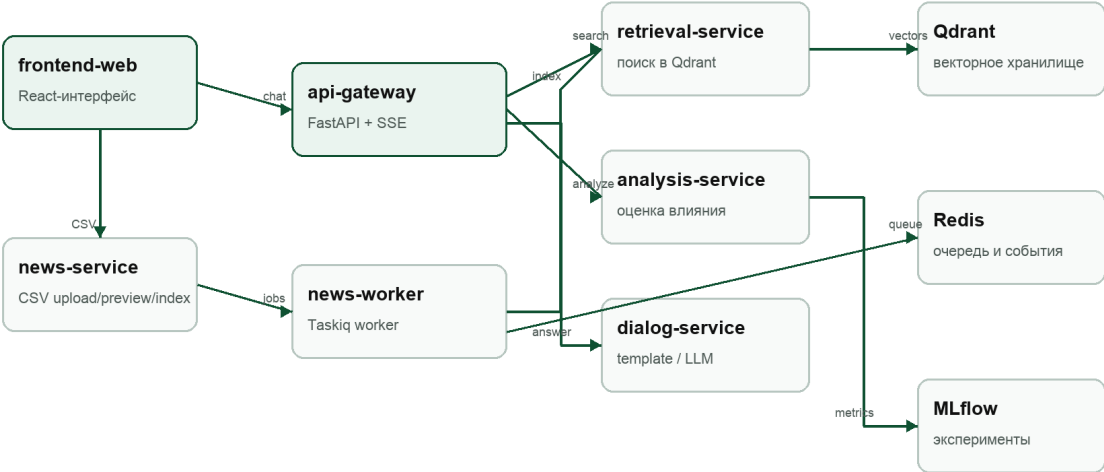
ML/NLP-задача	Роль в системе
Текстовое представление	TF-IDF и embeddings используются для поиска и классификации новостей.
Retrieval	Поиск релевантных документов формирует контекст для ответа.
Классификация влияния	Модель определяет позитивное, негативное или нейтральное влияние новости.
Генерация ответа	Dialog-service собирает ответ по найденным источникам через template или LLM-режим.
ML tracking	MLflow предусмотрен для фиксации экспериментов и артефактов моделей.

2. ПРОЕКТИРОВАНИЕ ДИАЛоговой СИСТЕМЫ

Система построена как локальный микросервисный стенд. Такой подход разделяет ответственность компонентов и позволяет показать полный pipeline обработки вопроса.

Архитектура диалоговой системы анализа экономических новостей

ML/NLP pipeline и production backend: загрузка CSV, поиск, анализ влияния, обучение и ответ



Сценарий: CSV upload -> активный датасет -> признаки и embeddings -> retrieval -> классификация -> ответ с источниками

Рисунок 1 - Архитектура диалоговой системы

Компонент	Ответственность
frontend-web	Веб-интерфейс чата, загрузки CSV, предпросмотра активного датасета и отображения источников.
api-gateway	Единая точка входа, оркестрация поиска, анализа и генерации ответа.
news-service	Загрузка CSV, хранение активного датасета, предпросмотр и запуск индексации новостей.
news-worker	Фоновая индексация через Taskiq и Redis.
retrieval-service	Индексация документов и поиск релевантных новостей в Qdrant.
analysis-service	Классификация влияния новости.
dialog-service	Формирование итогового ответа через template или LLM-режим.

Backend-сервисы используют слоистую архитектуру: domain, application, infrastructure, presentation и main. Интерфейсы портов описаны

через Protocol, а зависимости собираются через Dishka. Такое разделение сохраняет ясную границу между ML-логикой, доменными правилами и инфраструктурными адаптерами.

3. РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОЕКТА

Проект реализован в формате monorepo. Backend написан на Python с использованием FastAPI, asyncio и Granian. Межсервисные HTTP-вызовы выполняются через Zapro, настройки описываются через Pydantic Settings, структурные логи формируются через structlog.

Фоновые задачи вынесены в news-worker. Taskiq ставит задачи индексации, Redis используется как backend очереди, а FastStream публикует события. Этот вариант легче RabbitMQ и подходит для локального стенда.

News-service поддерживает загрузку пользовательского CSV через API и интерфейс. Загруженные файлы сохраняются в data/uploads, а выбранный файл фиксируется как активный датасет. После этого предпросмотр и индексация используют активный набор данных, при его отсутствии - демонстрационный CSV из data/raw.

Векторный поиск реализован через Qdrant. Для demo-режима поддерживаются статические embeddings, чтобы запуск не зависел от скачивания внешней модели.

ML/NLP-часть выделена в отдельные контракты и сервисы. Analysis-service получает текст новости, применяет выбранный режим анализа и возвращает класс влияния, confidence score и краткое объяснение на русском языке.

CLI prepare_dataset приводит внешний CSV к схемам приложения и обучения. После подготовки размеченного набора команды train-baseline, train-embedding, train-transformer и compare-models обучают и сравнивают классификаторы, а demo-up-trained запускает стенд с обученными артефактами.

Основной demo-режим tfidf-logreg выбран как объяснимый baseline. Он дополняется предусмотренными режимами embedding-logreg и tiny-transformer-classifier, чтобы в дальнейшем можно было сравнивать качество разных подходов без изменения внешнего API.

Dialog-service поддерживает template fallback и LLM-режим через OpenAI-compatible локальный сервер. Для легкого локального варианта предусмотрена модель Qwen3-0.6B-Instruct-GGUF через llama.cpp.

Часть	Технологии
ML: признаки текста	TF-IDF, статические embeddings
ML: подготовка данных	prepare_dataset, label-column, схемы для приложения и обучения
ML: классификация	Logistic Regression baseline, режимы embedding-logreg и tiny-transformer-classifier
NLP: retrieval/RAG	Qdrant, поиск контекста, ответ по источникам
NLP: генерация	Template fallback, OpenAI-compatible LLM через llama.cpp
ML tracking	MLflow
Backend	FastAPI, asyncio, Granian, Zapro
Архитектура	DDD, слои, Protocol, Dishka
Фоновые задачи и события	Taskiq, Redis, FastStream
Frontend и запуск	React, Docker Compose, Dockerfile, Justfile

4. ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ РЕЗУЛЬТАТОВ

Для проверки проекта подготовлены unit-тесты, тесты контрактов, API-тесты, frontend-тесты и demo smoke-сценарий. Проверка охватывает как инженерную связность сервисов, так и корректность ML/NLP-сценария: поиск источников, классификацию влияния и формирование ответа по найденному контексту.

Проверка	Назначение
git diff --check	Проверка форматных ошибок в diff.
docker compose config --quiet	Проверка корректности compose-конфигурации.
just demo-smoke	Проверка health endpoints, CSV preview, индексации, фоновой задачи, SSE и frontend.
frontend tests	Проверка React UI и клиентских API.
ML/NLP demo	Проверка найденных источников, score, класса влияния и итогового ответа.

Диалоговая система

Модель анализа

Лимит источников

Предпросмотр Индекс

Набор данных

Активен: uploaded.csv

Загрузить CSV

Вопрос

Что означает рост ВВП и снижение инфляции для рынка?

Ask

По вопросу найдены релевантные новости. Рост ВВП обычно указывает на расширение экономической активности, а снижение инфляции уменьшает давление на ставку и поддерживает ожидания рынка. Итоговая оценка строится на найденных источниках и не является финансовой рекомендацией.

Ход обработки

1. chat_started
2. search_started
3. sources_found
4. analysis_started
5. analysis_completed
6. answer_started
7. answer_completed
8. done

Источники

Центральный банк сохранил ставку
 Central Bank Brief
 score 0.79
 нейтральное

ВВП вырос быстрее ожиданий
 demo
 score 0.78
 позитивное

Безработица остается низкой
 Labor Statistics
 score 0.76
 нейтральное

Экспортные заказы сократились
 Trade Monitor
 score 0.72
 негативное

Рисунок 2 - Интерфейс демонстрационного сценария

Ручной сценарий демонстрации: запустить just demo-up, открыть <http://localhost:5173>, при необходимости загрузить CSV и убедиться, что он стал активным датасетом, выполнить предпросмотр CSV, индексировать новости и задать вопрос о росте ВВП и снижении инфляции.

Проверка обученных режимов выполняется через just prepare-dataset path/to/external.csv --label-column sentiment, затем just train-baseline, just train-embedding, just train-transformer, just compare-models и just demo-up-trained. В

интерфейсе последовательно выбираются режимы tfidf-logreg, embedding-logreg и tiny-transformer-classifier.

ЗАКЛЮЧЕНИЕ

В ходе работы была разработана локальная микросервисная диалоговая система для анализа экономических новостей. Система принимает вопрос пользователя, ищет релевантные новости, классифицирует их влияние и формирует ответ с указанием источников.

В работе объединены методы машинного обучения и инженерная реализация приложения. ML/NLP-результат включает подготовку датасета, обучение классификаторов, retrieval pipeline, классификацию влияния, поддержку разных режимов анализа и LLM-адаптер. Инженерный результат включает микросервисную архитектуру, асинхронные сервисы, загрузку и выбор активного CSV, фоновые задачи, SSE-интерфейс и воспроизводимый Docker Compose запуск.

Поставленная цель достигнута. Проект соответствует теме курсовой работы, поскольку реализует автоматический диалоговый сценарий на основе найденного контекста, анализа экономических новостей и подключаемой языковой модели.

Дальнейшее развитие может включать подключение реальных новостных API, расширение обучающих датасетов, сохранение истории диалогов в PostgreSQL, обучение дополнительных моделей анализа и сравнение качества template и LLM-режимов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. FastAPI Documentation. <https://fastapi.tiangolo.com/>
2. Granian Documentation. <https://github.com/emmett-framework/granian>
3. Pydantic Documentation. <https://docs.pydantic.dev/>
4. Dishka Documentation. <https://dishka.readthedocs.io/>
5. Taskiq Documentation. <https://taskiq-python.github.io/>

6. FastStream Documentation. <https://faststream.ag2.ai/>
7. Redis Documentation. <https://redis.io/docs/>
8. Qdrant Documentation. <https://qdrant.tech/documentation/>
9. MLflow Documentation. <https://mlflow.org/docs/latest/>
10. React Documentation. <https://react.dev/>
11. Docker Compose Documentation. <https://docs.docker.com/compose/>
12. Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software.
13. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
14. Vaswani A. et al. Attention Is All You Need.